



Universidad Autónoma
Metropolitana
Casa abierta al tiempo

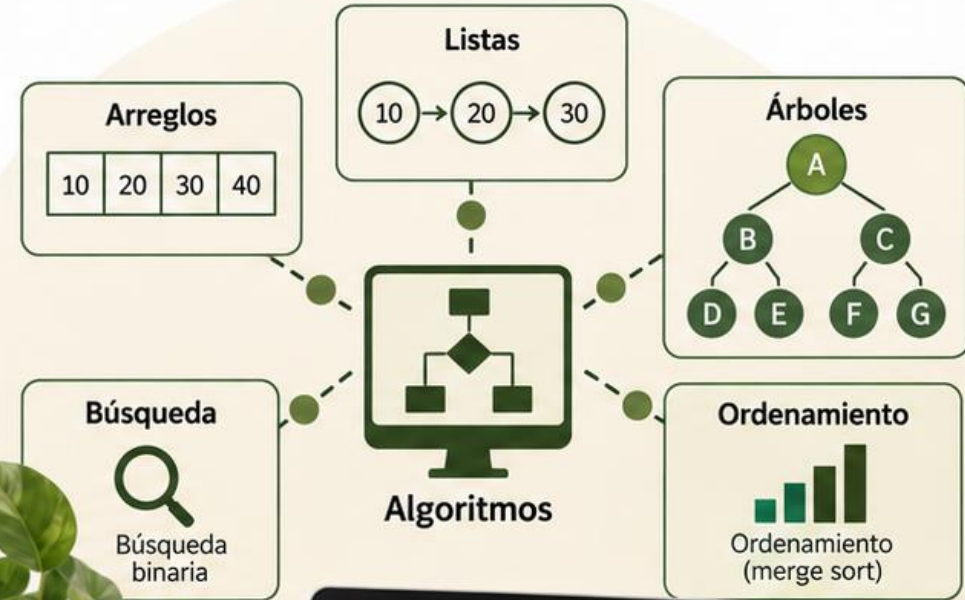


CONOCIMIENTO
QUE TRANSFORMA

Algoritmos y estructuras de datos



Ingeniería en Electrónica
Universidad Autónoma Metropolitana
Trimestre 26-O





Docente

Yanqueleth Molina-Tenorio

-  **Contacto:** yanqueleth@xanum.uam.mx
-  **Oficina:** T-305
-  **Asesorías:** 12:00–13:00 cualquier día de la semana
-  **Profesor – Investigador**



Vida académica

Docencia en algoritmos, estructuras de datos y programación.



Productos de investigación

Artículos, desarrollos tecnológicos y material didáctico.



Trabajo actual

Investigación y docencia en ciencia de la computación, con énfasis en algoritmos y procesamiento de datos.



Temas de interés

Algoritmos, estructuras de datos, programación, inteligencia computacional y educación en computación.



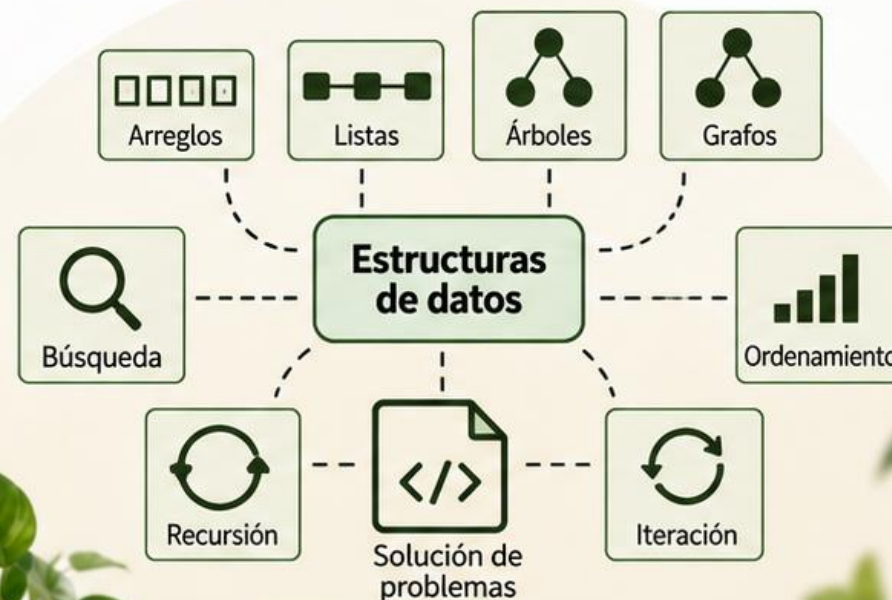
Objetivo



Implementar y aplicar diferentes tipos de datos abstractos en la solución de problemas.



Aplicar algoritmos iterativos y recursivos de búsqueda y ordenamiento para el diseño y desarrollo de programas.





Recursos



Clases



Prácticas de
laboratorio
"Champions League"



Exámenes
parciales



Exámenes
semanales



Grupo de difusión
de información



GitHub



Canal de YT



Algoritmos y estructuras de datos

Ingeniería en Electrónica
Universidad Autónoma Metropolitana
Trimestre 26-O





Horario y exámenes



Horario de clases

Miércoles y jueves: 10:00–11:30

📍 Salón: Sala F

Lunes: 10:00–12:00 hrs

📍 Salón: AT-114



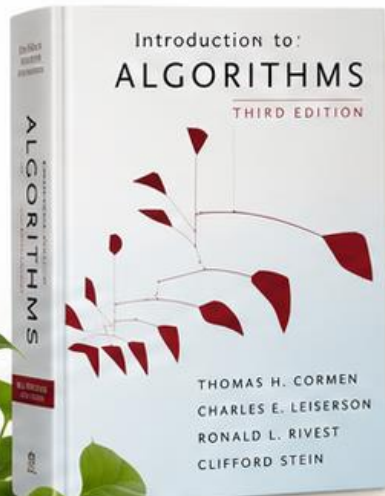
Exámenes

- | | |
|---------------------------------|---------------|
| 1. ^{er} examen parcial | Semanas 4–5 |
| 2. ^o examen parcial | Semana 9 |
| 3. ^{er} examen parcial | Semana 11 Bis |

i La aprobación del examen teórico es requisito para tener derecho al examen práctico.



Bibliografía



1

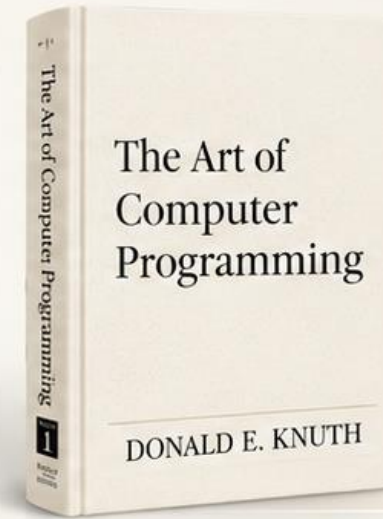
Cormen, Thomas H.;
Leiserson, Charles E.;
Rivest, Ronald L.;
Stein, Clifford.

**Introduction to
Algorithms.**
Third Edition.
Massachusetts Institute
of Technology, 2009.



2

Joyanes Aguilar, Luis.
**Fundamentos de
programación:**
algoritmos, estructuras
de datos y objetos.
McGraw-Hill, 2003.



3

Knuth, Donald.
**The Art of
Computer
Programming.**
Addison-Wesley
Professional, 1998.





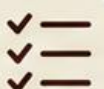
Evaluación



Tres exámenes parciales



60%



Exámenes semanales



20%



Laboratorio



20%



Nota 1:

Es indispensable cumplir al menos 70% de las actividades de laboratorio para tener derecho a calificación aprobatoria; de lo contrario corresponde NA.



Nota 2:

Pasados 10 minutos del inicio de clase, no se permitirá el ingreso al laboratorio/clase.



Calificación final

NA:	< 6.0
S:	6 – < 7.6
B:	7.6 – < 8.9
MB:	8.9 – 10.0

*De la teoría
a la práctica*





Reglamento interno

*Respetar las reglas,
construimos un mejor
aprendizaje*



- 1 Las aclaraciones de calificación deberán realizarse **el mismo día de la entrega.**



- 2 No podrá usarse **equipo computacional distinto al solicitado** por el docente durante la sesión académica.



- 3 Las tareas, proyectos, escenarios y demás trabajos se recibirán **únicamente en la fecha indicada.**



- 4 El uso de IA para resolver ejercicios, actividades o exámenes está **estrictamente prohibido** y podrá ser sancionado por el docente.

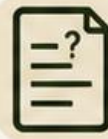




Exámenes semanales “Rapidines”



Cada semana se
dedicarán entre
10 y 20 minutos a
un examen corto.



Puede incluir
preguntas teóricas
o la resolución de
uno o dos problemas
vistos en clase.



El rapidín puede ser
de **preguntas abiertas**,
opción múltiple o
desarrollo de código.



La escala de
evaluación será
de **0 a 10**.





Prácticas de Laboratorio “Champions League”



Dinámica basada en enfrentamientos en la que se evaluará la **habilidad de programar** y la **resolución de problemas** en un **entorno competitivo**.

1



Preparación

Repaso de conceptos, herramientas y problemas de práctica.

2



Competencia en laboratorio

Enfrentamientos directos resolviendo problemas de programación.

3



Evaluación del desempeño

Resultados, retroalimentación y clasificación en la tabla del torneo.



Champions League



DINÁMICA:

- A cada alumno se le asigna un equipo de la Champions League.
- Los enfrentamientos se organizan según el número de alumnos (ida y vuelta o grupos, lo decide el docente).
- La práctica y el calendario de partidos se anuncian con anticipación.
- Los partidos no se limitan al laboratorio; puede haber más o menos prácticas según el avance del curso.



PROCESO:

- 1 Se anuncia la práctica.
- 2 Los alumnos preparan su código/solución antes del laboratorio.
- 3 En el laboratorio afinan detalles y juegan el partido.
- 4 Al terminar, pueden retirarse.



Más
práctica
mejores
soluciones



Champions League



Reglas

1



Que la terminal realice la tarea solicitada en la práctica.

2



Que el código sea eficiente.

3



Que el código corresponda a lo pedido en la práctica.

4



Que el código sea de autoría del alumno que presenta.

5



Que no se utilice IA para el desarrollo de la práctica.

6



Que el alumno sepa perfectamente qué realiza cada línea de código.

Buenas
prácticas,
grandes
programadores

Camino al éxito

- ☒ Códigos que funcionan
- ☒ Soluciones eficientes
- ☒ Trabajo original
- ☒ Aprendizaje real
- ☒ Sin uso de IA
- ☒ Conocimiento profundo



DISCIPLINA
CÓDIGO
CONOCIMIENTO
RESULTADOS



APRENDE
CODIFICA
PRACTICA
SUPERA

IDEAS
ALGORITMOS
PERSONAS
IMPACTO





Champions League



Puntuación



El alumno entrega al profesor la hoja "Enfrentamiento".



Se puede pactar el resultado antes de iniciar (victoria, derrota o empate).



Si el partido empieza, el resultado depende de lo que se juegue.



Victoria
→ 3 puntos



Empate pactado
→ 1 punto



Derrota
→ 0 puntos



Se consideran **todos los factores**: faltas, tarjetas, goles, porra, etc.



Más algoritmos,
más goles,
más futuro

Champions League

Goles y faltas



Goles = aciertos al presentar la práctica (código y terminal).



Faltas = errores al programar o mostrar en terminal.



Tarjetas:



Amarillas → errores conceptuales importantes (funciones, variables, teoría, etc.).



Rojas → faltas graves (copiar código, no autoría, cumplir menos del 50% de lo pedido).



*De la teoría
a la práctica*

Champions League

Lo evidente



Recomiendo ampliamente
que **juegue con estrategia.**

Piense seriamente cuándo le conviene
pactar un empate o cuándo le conviene
pactar una derrota.



Champions League

Puntos igual a calificación



Ejemplo:

- **MCI**: 5 ganados, 1 empate y 2 perdidos → 16 pts.
- **RMD**: 3 ganados, 4 empates y 1 perdido → 13 pts.
- **MAN**: 0 ganados, 1 empate y 7 perdidos → 1 pt.
- **BCN**: 8 ganados, 0 empates y 0 perdidos → 24 pts.

Equipo	PJ	PG	PE	PTS
MCI	8	5	1	16
RMD	8	3	4	13
MAN	8	0	1	1
BCN	8	8	0	24



Puntos = calificación



Relación lineal entre puntos y escala de 0 a 10



La calificación se obtiene mediante una **relación lineal** entre puntos y escala de 0 a 10.



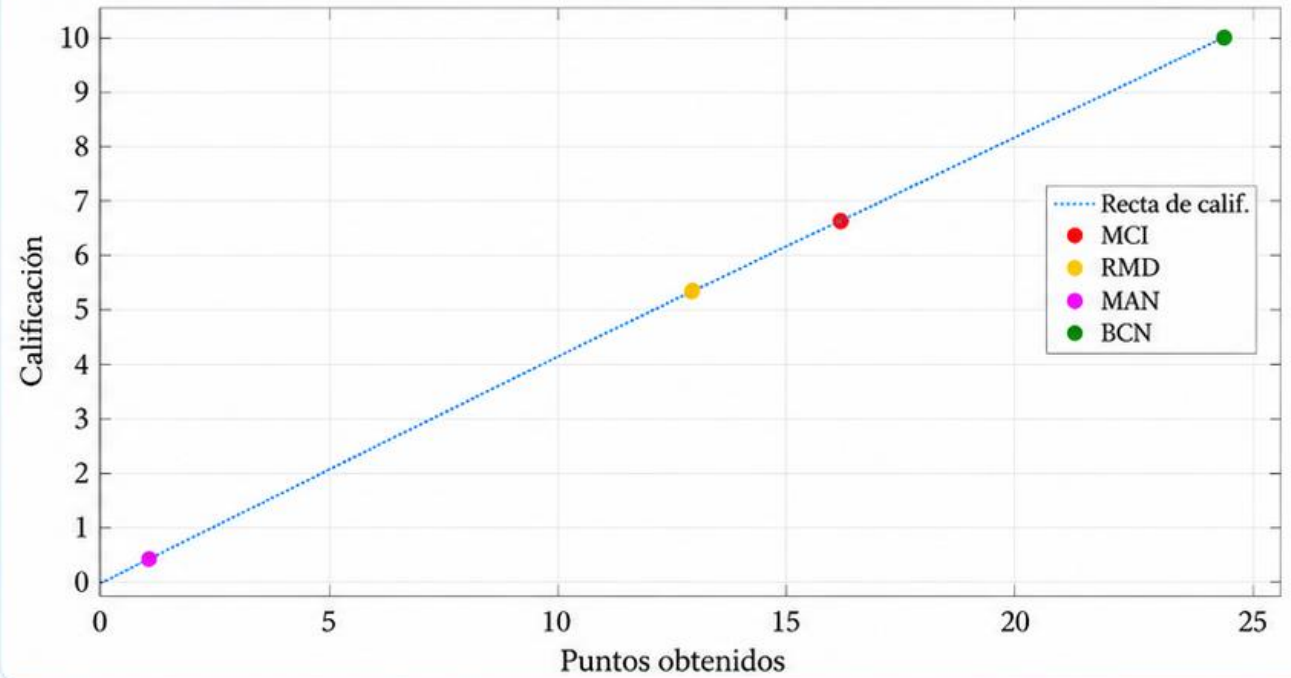
Si el máximo es 24 puntos, entonces **24 puntos equivalen a 10 de calificación**.



Ejemplos: MCI = 16 → 6.67,
RMD = 13 → 5.41,
MAN = 1 → 0.41, BCN = 24 → 10.



Relación entre puntos y calificación



Fórmula de conversión

Fórmula general

$$\text{Calificación} = \frac{\text{Puntos obtenidos} \times 10}{\text{Partidos jugados} \times 3}$$

Ejemplo

Si Partidos jugados = 8, entonces:

$$\text{Calificación} = \frac{\text{Puntos obtenidos} \times 10}{8 \times 3} = \frac{\text{Puntos obtenidos} \times 10}{24}$$

$$\text{Calificación} = 0.4167 \times \text{Puntos obtenidos}$$

Mismos
puntos,
grandes
logros



Champions League

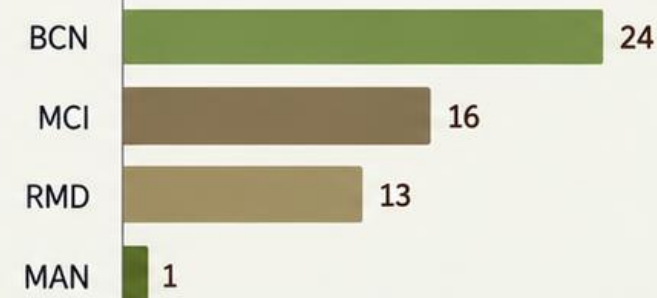


Puntos igual a calificación

Ejemplo de conversión de puntos a calificación del laboratorio.

Equipo	PTS	CAL	% total
MCI	16	6.67	13.34
RMD	13	5.41	10.80
MAN	1	0.41	0.82
BCN	24	10.00	20.00

Ranking por puntos



De los datos
a la práctica





Sorteo

Asignación de equipos y enfrentamientos





Semana de regularización

- ✓ Tipos de datos
- ✓ Comparación y asignación
- ✓ Condicional *if*
- ✓ Ciclos iterativos
- ✓ Estructura arreglo
- ✓ Funciones y procedimientos
- ✓ Punteros



Tipos de datos

Algoritmos y estructuras de datos

Ingeniería en Electrónica
Universidad Autónoma Metropolitana
Trimestre 26-O

- ✓ Un tipo de dato define qué clase de valor puede almacenar una variable.
- ✓ También determina qué operaciones se pueden realizar con ese valor.
- ✓ Elegir el tipo correcto ayuda a ahorrar memoria y evitar errores.
- ✓ En programación se usan tipos básicos y, más adelante, tipos estructurados.



Buenos
datos,
mejores
soluciones



Tipos de datos

Ejemplo resuelto

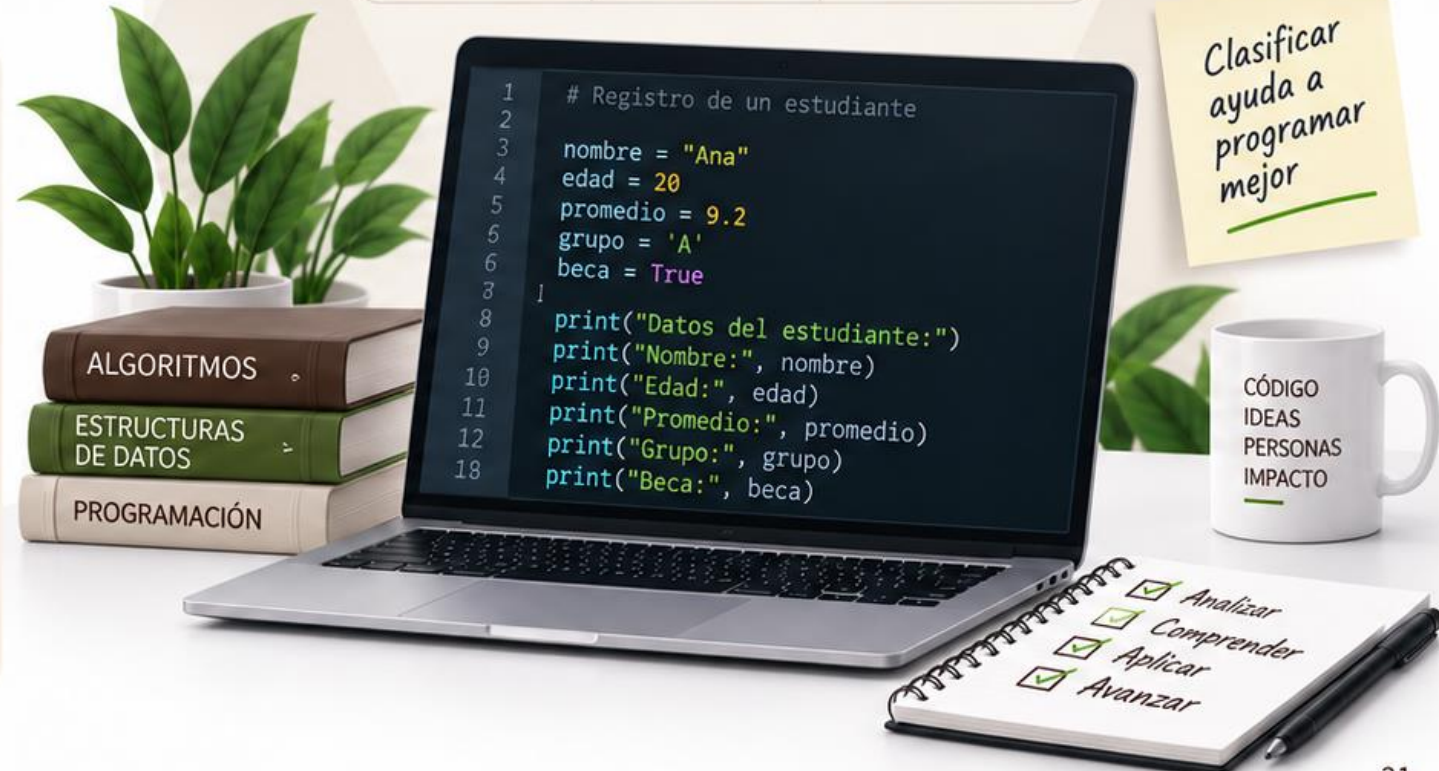
Algoritmos y estructuras de datos
Ingeniería en Electrónica
Universidad Autónoma Metropolitana
Trimestre 26-O

Registro de un estudiante

Analicemos las siguientes variables y el tipo de dato que les corresponde.

- 1 **nombre = "Ana"**
Cadena, porque almacena texto.
- 2 **edad = 20**
Entero, porque no tiene decimales.
- 3 **promedio = 9.2**
Real, porque tiene decimales.
- 4 **grupo = 'A'**
Carácter, porque es un solo símbolo.
- 5 **beca = True**
Booleano, porque solo puede ser verdadero o falso.

Variable	Valor	Tipo
nombre	"Ana"	Cadena
edad	20	Entero
promedio	9.2	Real
grupo	'A'	Carácter
beca	True	Booleano



Clasificar ayuda a programar mejor

Tipos de datos

Ejercicios

Algoritmos y estructuras de datos
Ingeniería en Electrónica
Universidad Autónoma Metropolitana
Trimestre 26-O

1 Ejercicio 1

Indica qué tipo de dato corresponde a cada valor:

- 1 25
- 2 3.5
- 3 'M'
- 4 "UAM"
- 5 False

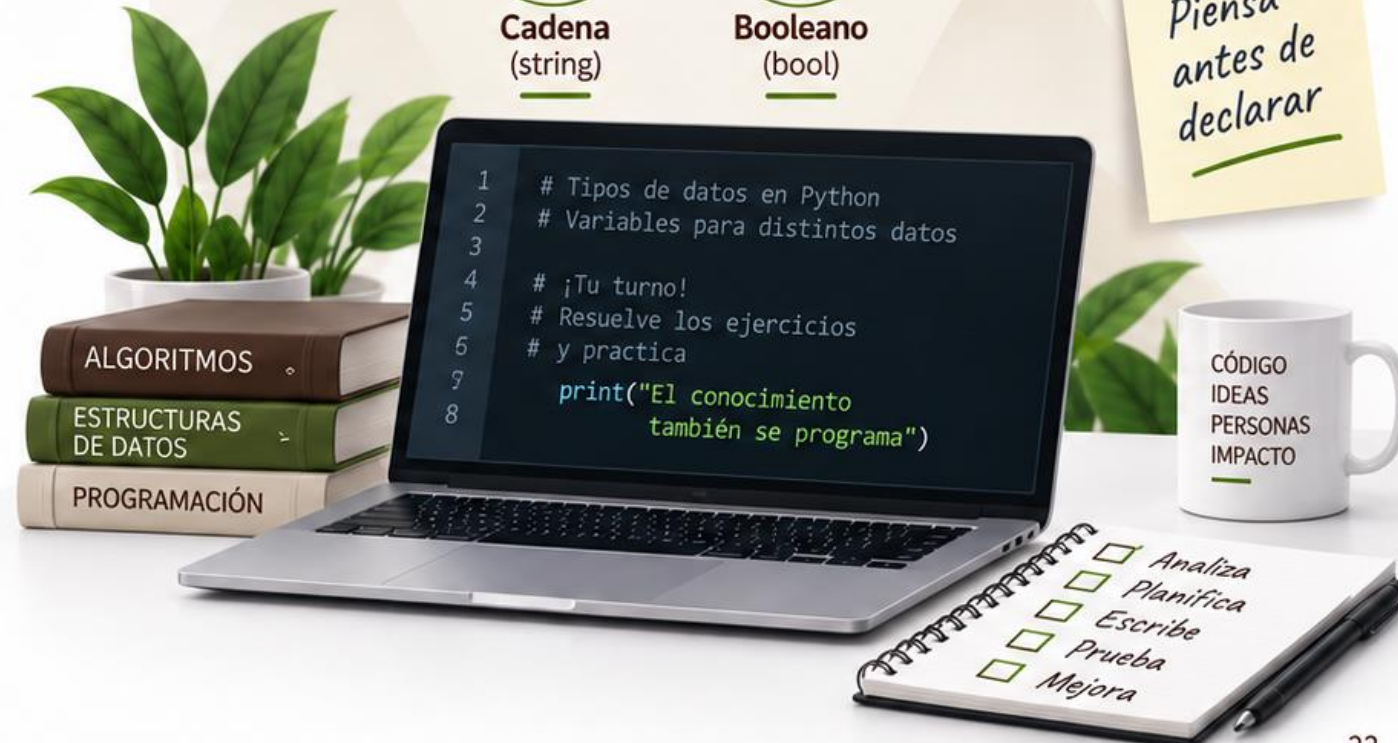
2 Ejercicio 2

Escribe una posible declaración de variable para cada caso:

- 1 edad del alumno
- 2 estatura
- 3 inicial del nombre
- 4 carrera
- 5 inscrito



Piensa
antes de
declarar



Comparación y asignación

Algoritmos y estructuras de datos

Ingeniería en Electrónica

Universidad Autónoma Metropolitana

Trimestre 26-O

Asignación



Guardar un valor
en una variable.

```
x = 5  
nombre = "Ana"
```

Comparación



Verificar la relación
entre dos valores.

```
x == 5    → Verdadero  
x > 3     → Verdadero  
x != 2    → Falso
```



Asignación significa guardar un valor
en una variable.



Se usa el operador =.



Ejemplo: `x = 5`.



Comparación significa verificar la relación
entre dos valores.



Produce un resultado booleano: verdadero o falso.



Operadores comunes: `==`, `!=`, `>`, `<`, `>=`, `<=`.



Pequeñas
instrucciones,
grandes
resultados



Comparación y asignación

Ejemplo resuelto

Algoritmos y estructuras de datos
Ingeniería en Electrónica
Universidad Autónoma Metropolitana
Trimestre 26-O

Promedio de dos calificaciones

- 1 Asignamos valores:
cal1 = 8 y cal2 = 9.
- 2 Calculamos el promedio:
 $\text{promedio} = (\text{cal1} + \text{cal2}) / 2 = 8.5$.
- 3 Comparamos:
 $\text{promedio} \geq 6$.
- 4 Resultado de la comparación:
verdadero.
- 5 Conclusión:
el alumno aprueba.

Valores de las variables

Variable	Valor	Significado
cal1	8	Primera calificación
cal2	9	Segunda calificación
promedio	8.5	Promedio de las dos calificaciones

Misma
base,
mejores
posibilidades



✓ Aprobado

Comparación y asignación

Ejercicios

Algoritmos y estructuras de datos
Ingeniería en Electrónica
Universidad Autónoma Metropolitana
Trimestre 26-O

1 Ejercicio 1

Dado $a = 12$ y $b = 7$, indica el resultado de las comparaciones:

- $a > b$
- $a < b$
- $a == b$
- $a != b$
- $a >= b$

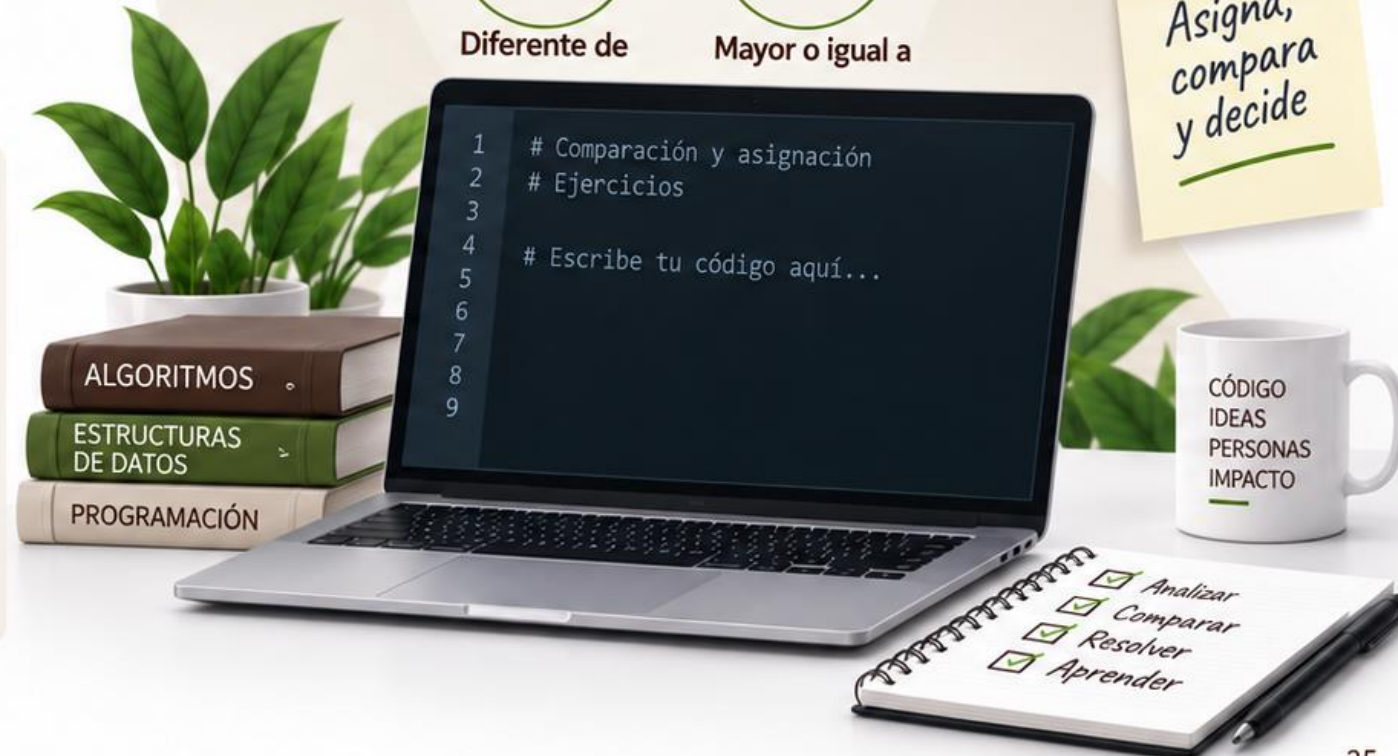
2 Ejercicio 2

Escribe un pequeño fragmento de código donde:

- `nombre = "Luis"`
- `edad = 19`
- y luego compara si `edad >= 18`.



Asigna,
compara
y decide



Condicional *if*

Algoritmos y estructuras de datos

Ingeniería en Electrónica

Universidad Autónoma Metropolitana

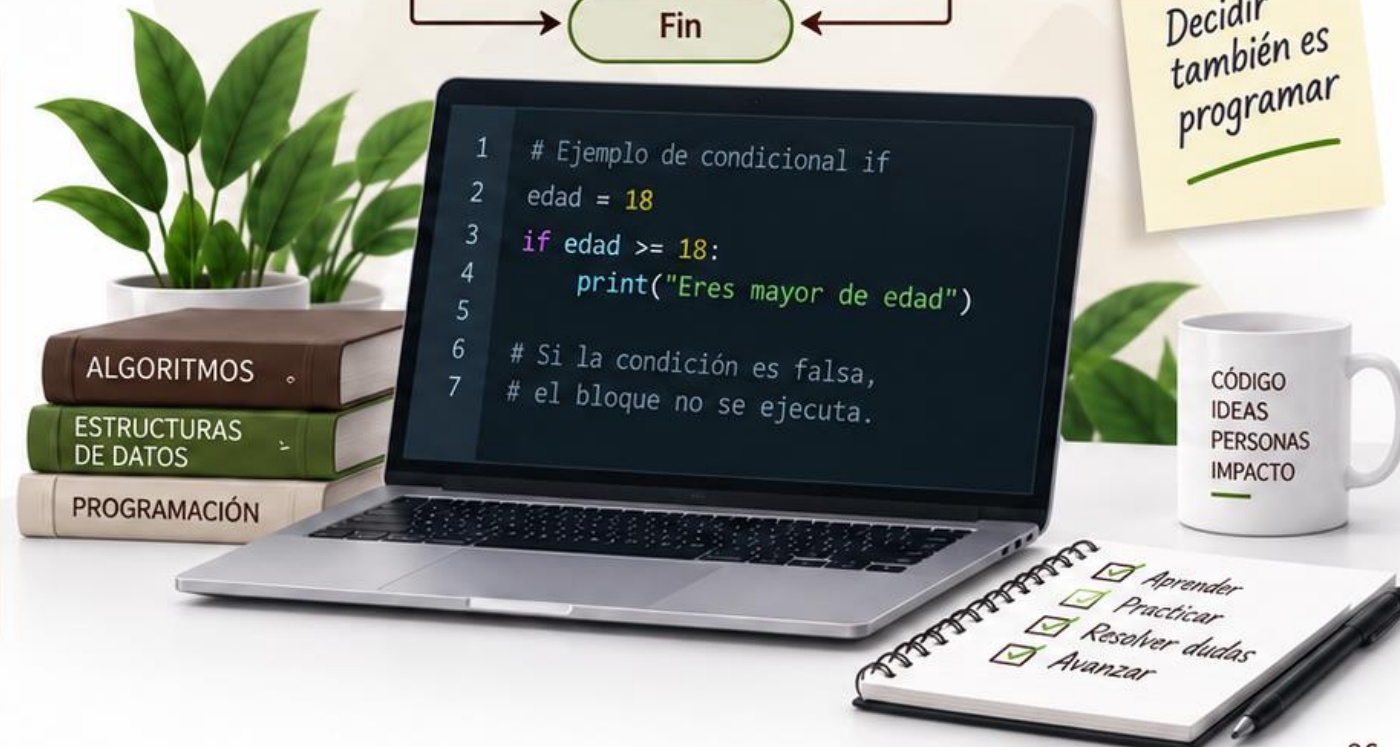
Trimestre 26-O

- ✓ La estructura *if* permite tomar decisiones.
- ✓ Ejecuta un bloque de instrucciones solo si la condición es verdadera.
- ✓ Si la condición no se cumple, el bloque no se ejecuta.
- ✓ Puede complementarse con *else* y *elif*.
- ✓ Sintaxis básica:

```
if condicion:
    instruccion
```



Decidir
también es
programar



Condicional *if*

Ejemplo resuelto

Algoritmos y estructuras de datos
Ingeniería en Electrónica
Universidad Autónoma Metropolitana
Trimestre 26-O

Determinar si un alumno aprueba

- 1 Se asigna la variable calificacion = 8.3.
- 2 Se evalúa la condición: calificacion $\geq$ 6.
- 3 Como la condición es verdadera, se ejecuta el bloque del *if*.
- 4 El programa muestra: "Aprobado".
- 5 Si la condición fuera falsa, mostraría: "No aprobado".



Condicional *if*

Ejercicios

Algoritmos y estructuras de datos
Ingeniería en Electrónica
Universidad Autónoma Metropolitana
Trimestre 26-O

1 Ejercicio 1

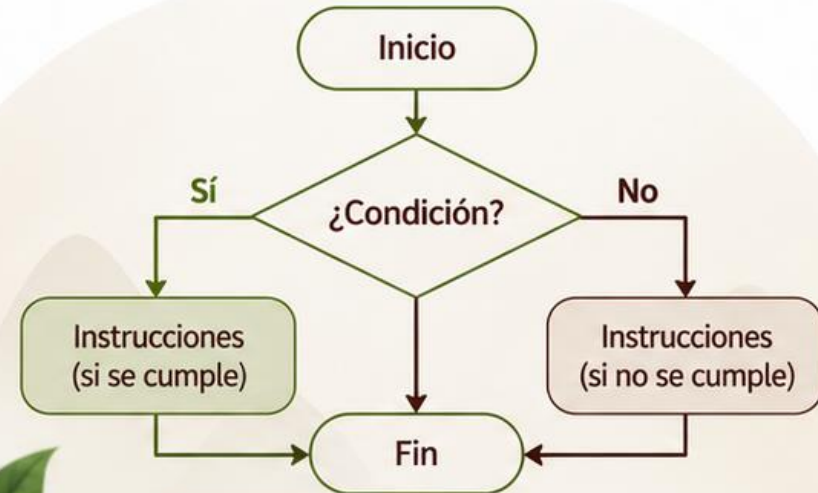
Escribe un programa que lea un número y muestre si es positivo o negativo.

2 Ejercicio 2

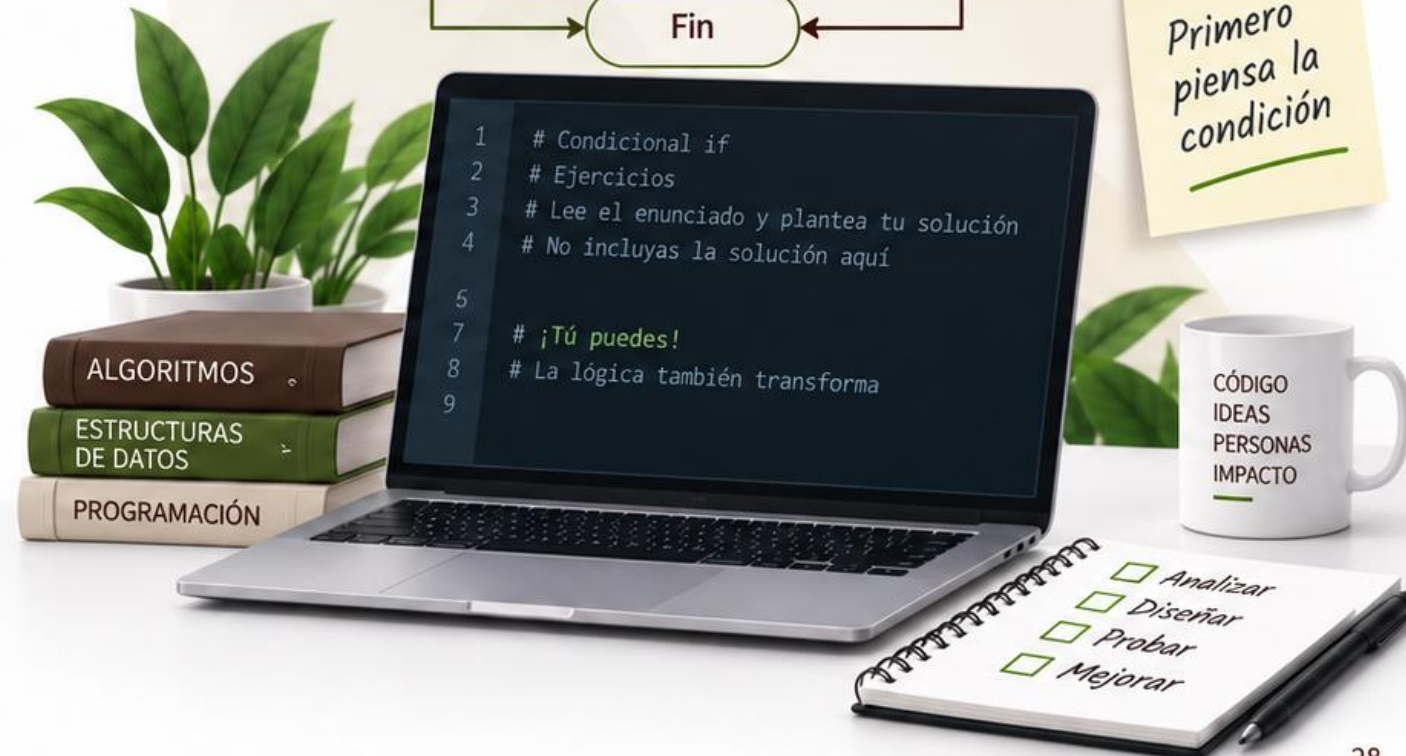
Escribe un programa que lea el monto de una compra y muestre "Descuento" si el monto es mayor a 1000; en caso contrario, muestre "Sin descuento".



No resuelvas aquí, solo plantea la solución.



Primero
piensa la
condición



Ciclos iterativos

- ✓ Un ciclo iterativo es una estructura que repite instrucciones varias veces.
- ✓ Permite evitar escribir código repetido.
- ✓ Estructuras comunes: **for** y **while**.
- ✓ Se usa con un contador o con una condición.
- ✓ Ejemplos de uso: contar, sumar, recorrer datos.

- Algoritmos y estructuras de datos
- Ingeniería en Electrónica
- Universidad Autónoma Metropolitana
- Trimestre 26-O

Flujo de un ciclo



Ejemplo con for

```
for i in range(5):
    print(i)
# Imprime 0, 1, 2, 3, 4
```

Ejemplo con while

```
i = 0
while i < 5:
    print(i)
    i = i + 1
# Imprime 0, 1, 2, 3, 4
```

Itera
Aprende
Programa
Avanza



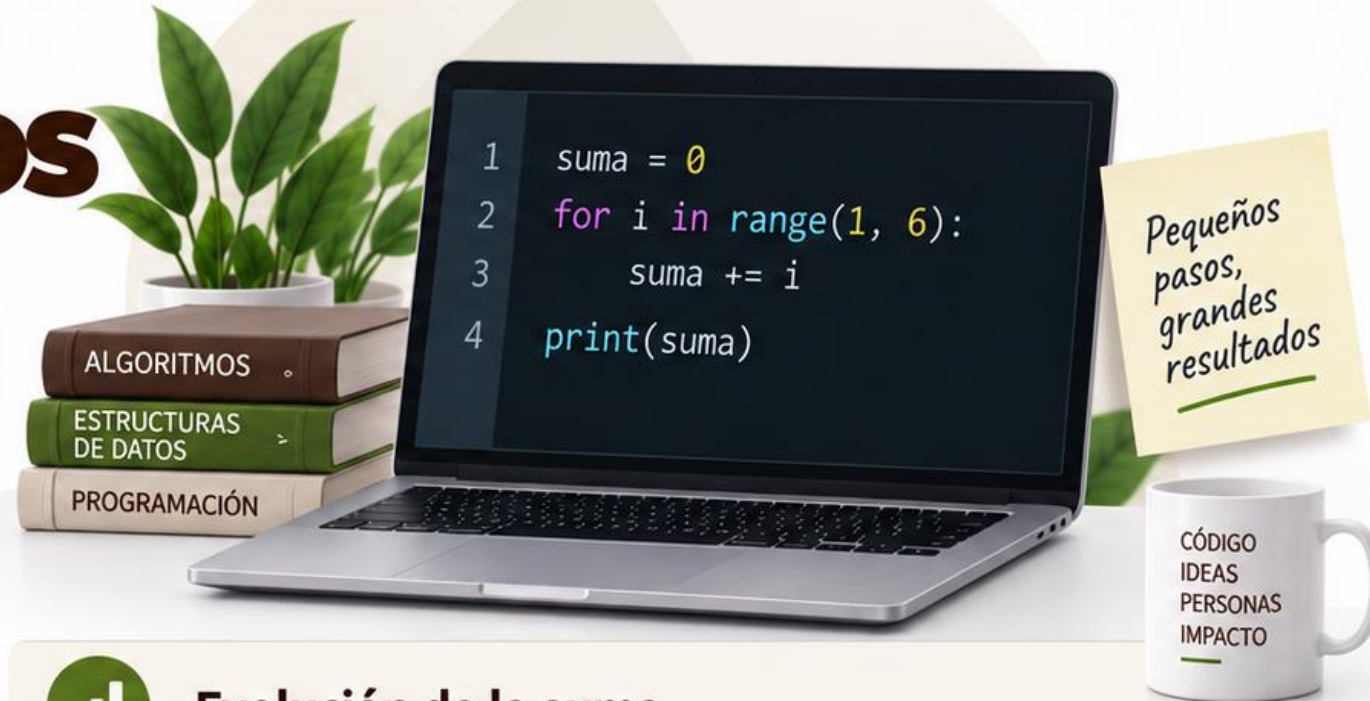
Ciclos iterativos

Ejemplo resuelto



Sumar los números del 1 al 5

- 1 Iniciar suma = 0.
- 2 Repetir con **for** desde 1 hasta 5.
- 3 Agregar cada valor a suma.
- 4 Al final suma = 15.
- 5 Mostrar resultado.



Evolución de la suma

i	1	2	3	4	5
Suma acumulada	1	3	6	10	15

Ciclos iterativos

Ejercicios

1

Escribe un programa que imprima los números del 1 al 10 usando un ciclo **for**.

2

Escribe un programa que lea un número n y calcule la suma de los números desde 1 hasta n usando un ciclo **while**.



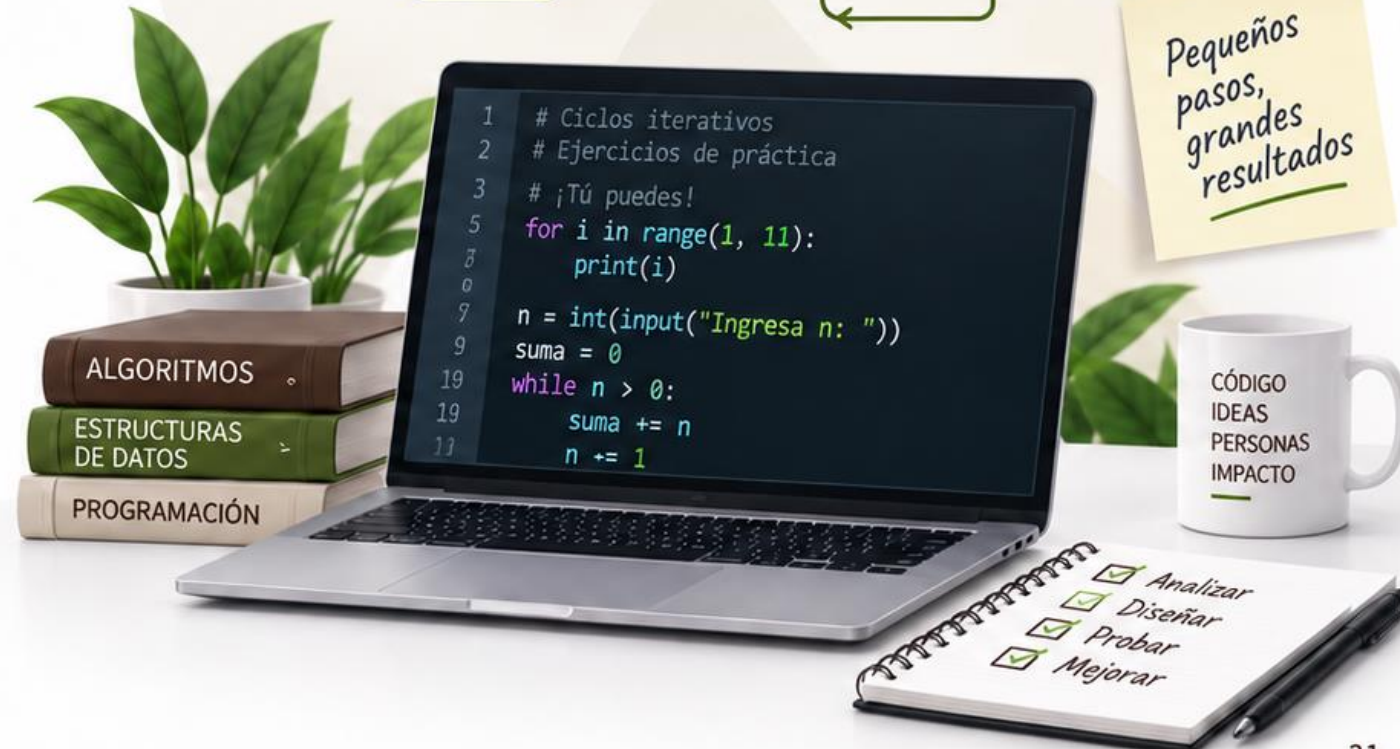
No resuelvas aquí, solo plantea la solución.

Ciclo for

```
for (i = 1; i <= n; i++)
{
    ...
}
```

Repite

Ciclo while



Estructura arreglo

- ✓ Un **arreglo** almacena varios valores en una sola estructura.
- ✓ Sus elementos se acceden por **posición** o **índice**.
- ✓ Permite **organizar datos relacionados**.
- ✓ Permite **leer, escribir y recorrer** sus valores.
- ✓ Es útil para **listas de calificaciones, edades, mediciones**, entre otros.

Arreglo de calificaciones



Varios
valores,
una sola
estructura



Estructura arreglo

Ejemplo resuelto

Calificaciones de un alumno

- 1 El arreglo tiene 4 elementos.
- 2 El primer valor es 8.
- 3 El último valor es 10.
- 4 La suma es 34.
- 5 El promedio es 8.5.

Arreglo: calificaciones

0	1	2	3
8	9	7	10

calificaciones = [8, 9, 7, 10]

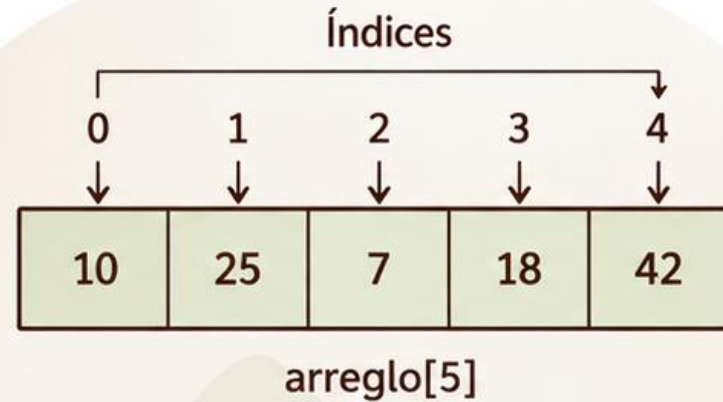
Misma
base,
mayores
posibilidades

```
1 # Calificaciones de un alumno
2 calificaciones = [8, 9, 7, 10]
3
4 suma = sum(calificaciones)
5 promedio = suma / len(calificaciones)
6
7 print("Calificaciones:", calificaciones)
8 print("Suma:", suma)
9 print("Promedio:", promedio)
```

CÓDIGO
IDEAS
PERSONAS
IMPACTO

- ☒ Analizar
- ☒ Implementar
- ☒ Verificar
- ☒ Aprender

Estructura arreglo Ejercicios



1

Declara un arreglo con 5 números enteros e imprime el primer y el último elemento.

2

Declara un arreglo con 4 calificaciones y calcula el promedio.



No resuelvas aquí, solo plantea la solución.



Funciones y procedimientos

- ✓ Una **función** es un bloque reutilizable que puede recibir parámetros y regresa un valor.
- ✓ Un **procedimiento** es similar, pero principalmente realiza acciones y puede no devolver un valor.
- ✓ Ayudan a organizar el código.
- ✓ Evitan la repetición.
- ✓ Mejoran la legibilidad y el mantenimiento del programa.



Función

Recibe parámetros y regresa un valor



Procedimiento

Realiza acciones y puede no regresar un valor

Código más claro
ideas más grandes

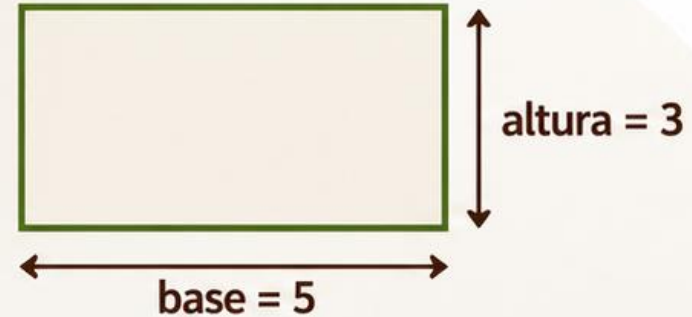


Funciones y procedimientos

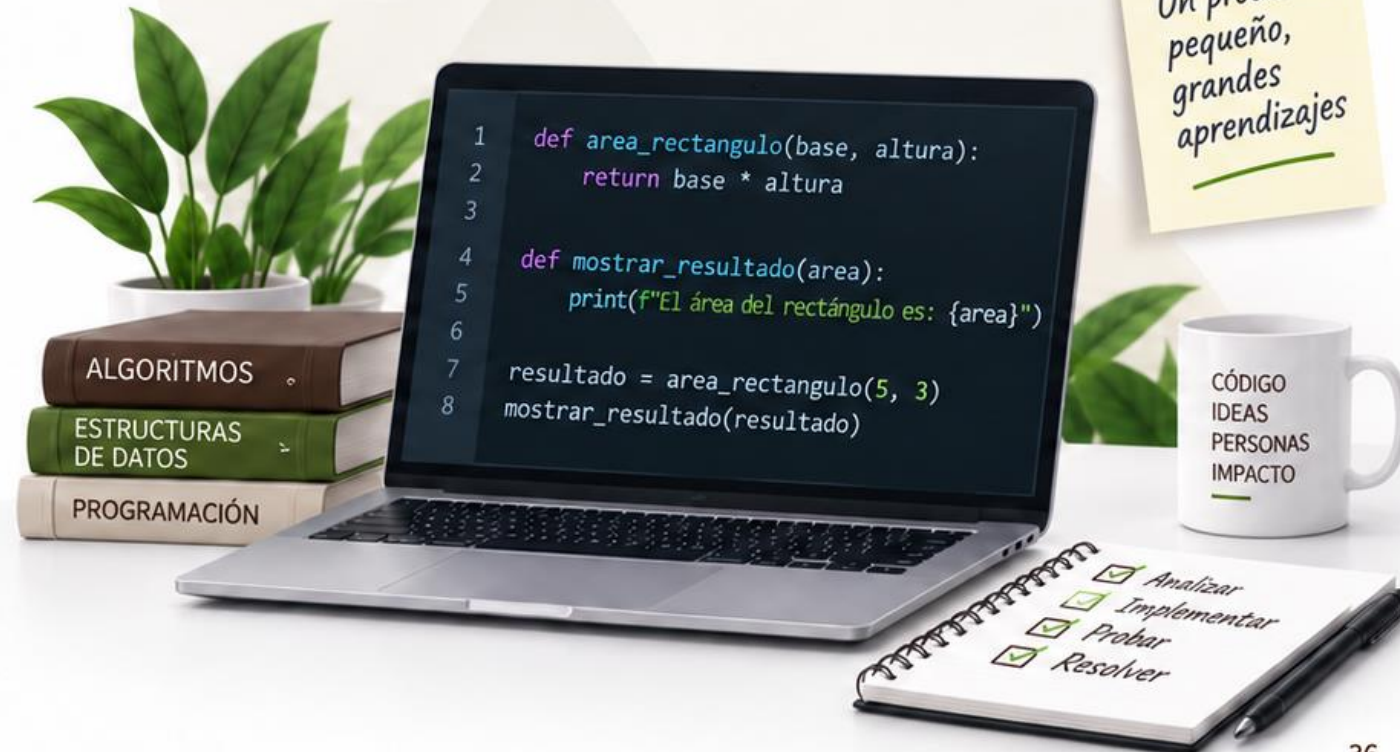
Ejemplo resuelto

Calcular el área de un rectángulo

- 1 La función recibe base y altura.
- 2 Calcula $\text{area} = \text{base} * \text{altura}$.
- 3 Regresa el resultado.
- 4 Si $\text{base} = 5$ y $\text{altura} = 3$, el área es 15.



Un problema
pequeño,
grandes
aprendizajes



Funciones y procedimientos

Ejercicios

1 Escribe una función llamada **cuadrado(numero)** que reciba un número y regrese su cuadrado.

2 Escribe un procedimiento llamado **saludar(nombre)** que muestre un mensaje de bienvenida.



No resuelvas aquí, solo plantea la solución.



Punteros Ejercicios

Algoritmos y estructuras de datos
Ingeniería en Electrónica
Universidad Autónoma Metropolitana
Trimestre 26-O

1

Declara una variable entera $x = 8$, crea un puntero p que apunte a x e imprime el valor de x , la dirección de x y el valor apuntado por p .

2

Escribe un programa que declare dos variables enteras a y b , use punteros para intercambiar sus valores y después los muestre en pantalla.



No resuelvas aquí, solo plantea la solución.

Memoria

Dirección	Valor
0x1000	8
0x1004	?
0x1008	?
0x100C	?

p apunta a x

x
8
0x1000

$\&x$

(dirección de x)

p
0x1000
0x1008

$*p$

(valor apuntado)

Primero
identifica
la dirección.

```
1 # Punteros
2 # Ejercicios
3
4 # Practica y refuerza
5 # tus conocimientos
6 # sobre direcciones de memoria
7 # y el operador de indirección
8 # ¡Tú puedes!
```

ALGORITMOS

ESTRUCTURAS
DE DATOS

PROGRAMACIÓN

CÓDIGO
IDEAS
PERSONAS
IMPACTO

☒ Revisar
☒ Practicar
☒ Resolver dudas
☒ Avanzar

Punteros Ejercicios

Algoritmos y estructuras de datos
Ingeniería en Electrónica
Universidad Autónoma Metropolitana
Trimestre 26-O

1

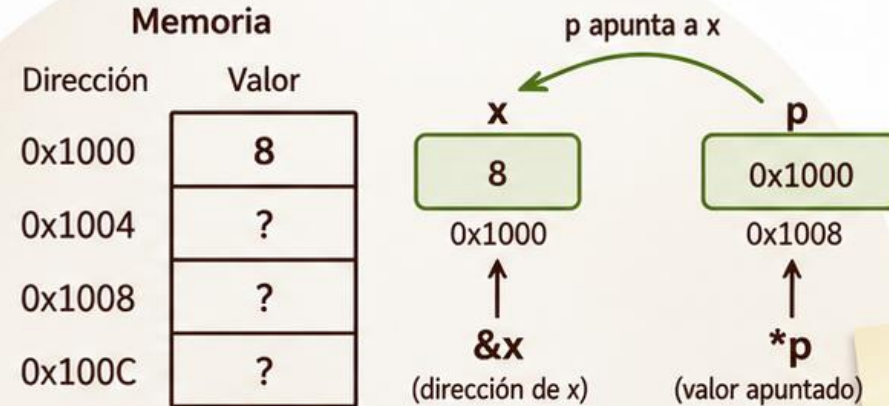
Declara una variable entera $x = 8$, crea un puntero p que apunte a x e imprime el valor de x , la dirección de x y el valor apuntado por p .

2

Escribe un programa que declare dos variables enteras a y b , use punteros para intercambiar sus valores y después los muestre en pantalla.



No resuelvas aquí, solo plantea la solución.



Primero
identifica
la dirección.



Punteros Ejercicios

Algoritmos y estructuras de datos
Ingeniería en Electrónica
Universidad Autónoma Metropolitana
Trimestre 26-O

1

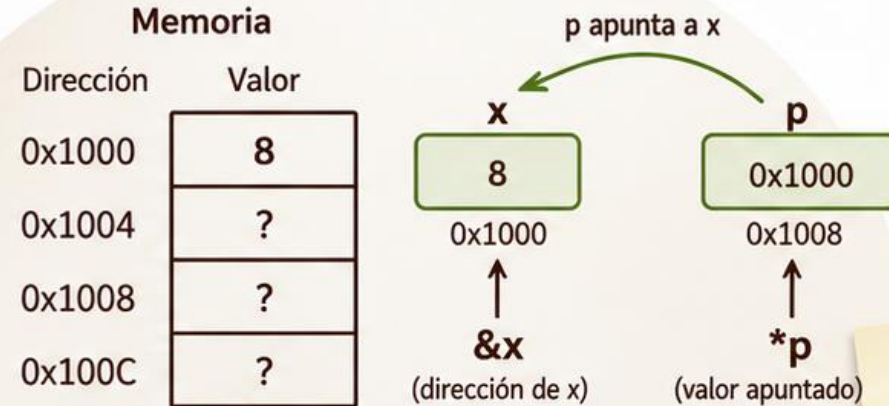
Declara una variable entera $x = 8$, crea un puntero p que apunte a x e imprime el valor de x , la dirección de x y el valor apuntado por p .

2

Escribe un programa que declare dos variables enteras a y b , use punteros para intercambiar sus valores y después los muestre en pantalla.



No resuelvas aquí, solo plantea la solución.



Primero
identifica
la dirección.

